

LLMs as a reader

How to use them to extract Topics from Text

2nd Workshop on Frontiers in Measurement and Survey Methods
MeToD Project · University of Calabria · 19 May 2026

What this talk covers

the measurement problem

traditional topic modelling

TRADITIONAL

LLM as embedding space

LLM reads end-to-end

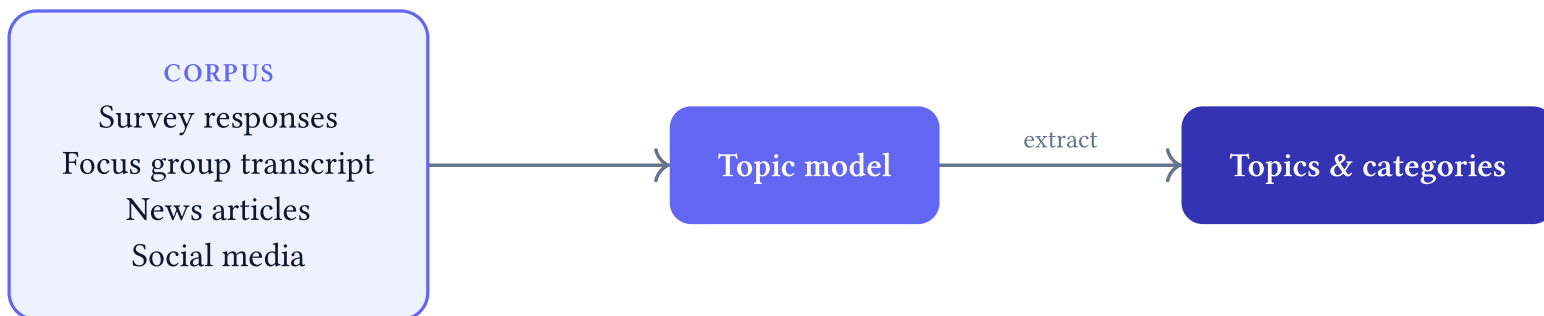
when to use what

LLM-POWERED

From LLM as a component to LLM as the whole reader.

Topic modeling: from text to data

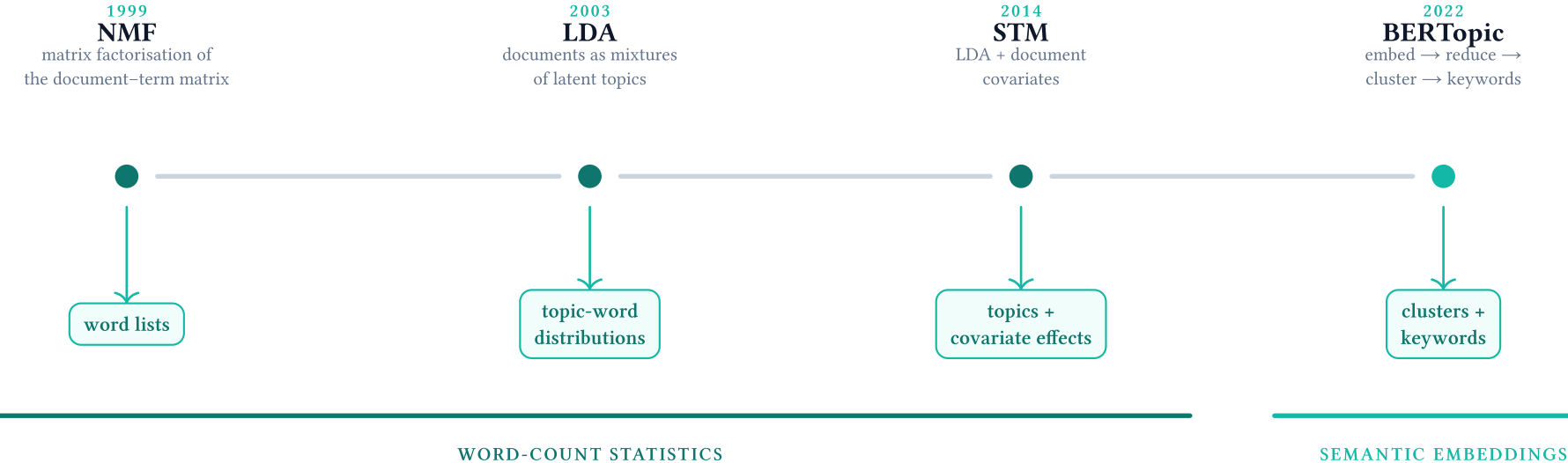
Raw text at scale is essentially opaque, humans can't read a million documents, and standard statistics can't summarize meaning.



Topic modeling is the task of discovering or assigning coherent, interpretable themes (*topics*) that organize a collection of documents.

Topics may be word distributions (traditional) or named labels with descriptions (LLM-based). Either way, you get categories you can **count, compare, and reason about**.

Topic modelling, two decades on



Two decades of evolution: from **word-count statistics** (NMF, LDA, STM) to **semantic embeddings** (BERTopic). The representation of text itself keeps getting richer.

Lee & Seung 1999, *Nature* · Blei, Ng & Jordan 2003, *JMLR* · Roberts et al. 2014, *AJPS* · Grootendorst 2022, *arXiv*

How BERTopic works

BERTopic isn't one model, it's a **modular pipeline**.

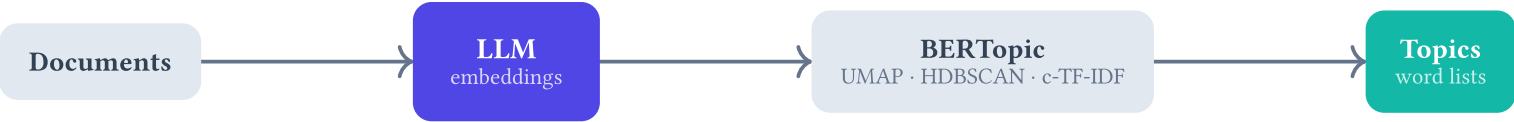


- **Embeddings:** Turn documents into vector representations – [docs](#)
- **Dimensionality reduction:** Compress the embedding space (UMAP, PCA, t-SNE) – [docs](#)
- **Clustering:** Group similar documents (HDBSCAN, k-means, BIRCH) – [docs](#)
- **Tokenizer:** Split each cluster back into tokens for the weighting step – [docs](#)
- **Weighting scheme:** Surface the most distinctive words per cluster (c-TF-IDF, BM25) – [docs](#)

Every stage is swappable. Today we focus on **one stage: the embeddings**.

Two ways to bring LLMs in

Approach 1 LLM as a component, replace one box



Approach 2 LLM as the whole reader, replace the pipeline



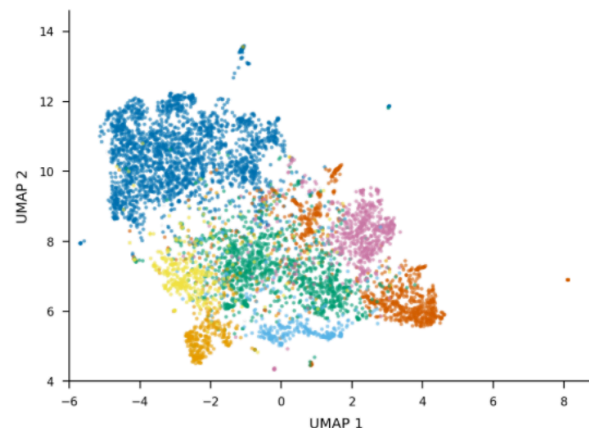
Approach 1: LLM as the embedding space

Keep BERTopic. Swap its embedding model for **LLM embeddings** — everything downstream is unchanged.



Embedding space: High-dimensional map of meaning, built by a transformer pretrained on massive corpora.

Each document becomes a vector; semantic similarity becomes geometric distance, so synonyms and paraphrases cluster together, *something word-count methods could never do*.



Reimers & Gurevych 2019, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, EMNLP · the foundation of the SBERT embeddings BERTopic uses by default.

Approach 1: what changes, what stays

LLM embeddings sharpen the **input**. Everything downstream (clustering, word lists, manual labelling) is still **classic BERTopic**.

WHAT IMPROVES

Better embeddings help with

- short, noisy text (tweets, one-liners)
- multilingual corpora
- domain-specific meaning
- semantic clusters, not keyword clusters

STILL YOUR JOB

The rest of the pipeline is unchanged

- tune UMAP / HDBSCAN, handle outliers
- topics still come out as **word lists**
- you still **read and label** them
- the LLM upgraded the *representation*, not the *reading*

Approach 1 in practice — load the corpus

Datasets live on the [Hugging Face Hub](#). Pandas reads them directly via the `hf://` URI scheme — no download script, no caching code.

```
import pandas as pd

splits = {"train": "train.jsonl", "test": "test.jsonl"}
df = pd.read_json(
    "hf://datasets/SetFit/20_newsgroups/" + splits["train"],
    lines=True,
)
df.head()
```

One line in, a tidy DataFrame out — `text`, `label`, `label_text`. The full pipeline (encode with `bge-base` → BERTopic) lives in the [companion notebook](#).

Dataset: [SetFit/20_newsgroups](#) on Hugging Face · original corpus: Lang 1995, *Newsweeder: Learning to filter netnews*, ICML

Vanilla BERTopic — the baseline

Two lines. No encoder choice, no UMAP/HDBSCAN tuning — BERTopic falls back to its built-in defaults.

```
from bertopic import BERTopic

topic_model = BERTopic(verbose=True)
topics, _ = topic_model.fit_transform(docs)
topic_model.get_topic_info().head(10)
```

What you get out of the box

- encoder: `all-MiniLM-L6-v2` (22 M params, 384-d)
- dim. reduction: UMAP, `n_neighbors=15`, `n_components=5`
- clustering: HDBSCAN, `min_cluster_size=10`
- representation: c-TF-IDF on the bag of words

Why this is the right baseline

- it's what every BERTopic tutorial starts with
- every later improvement (LLM encoder, tuned HDBSCAN) is measured *against* this
- on 20 Newsgroups: many tiny topics, ~30 % outliers — visible in the companion notebook

Tuned BERTopic — step 1: a stronger encoder

Swap the default MiniLM for a modern LLM-grade sentence encoder. Same API, much sharper embedding space.

```
import torch
from sentence_transformers import SentenceTransformer

device      = "cuda" if torch.cuda.is_available() else "cpu"
encoder      = SentenceTransformer("BAAI/bge-base-en-v1.5", device=device)
embeddings   = encoder.encode(docs, batch_size=128, convert_to_numpy=True)
```

Why this encoder

- default `all-MiniLM-L6-v2` is the smallest SBERT (22 M, 384-d)
- `bge-base-en-v1.5` — 110 M params, 768-d, modern (2023)
- consistently strong on the MTEB leaderboard among base-sized models

GPU notes

- 8 GB VRAM is plenty for ~4 k docs at `batch_size=128`
- ~10–15 s end-to-end on a laptop RTX 4070
- want more quality? `bge-large-en-v1.5` (335 M, 1024-d) — same API, ~3× slower

Tuned BERTopic — step 2: tune the clustering

Pass the embeddings into BERTopic with explicit UMAP and HDBSCAN — and lift `min_cluster_size` so HDBSCAN stops shattering each newsgroup.

```
from umap import UMAP
from hdbscan import HDBSCAN
from bertopic import BERTopic

umap_model = UMAP(n_neighbors=15, n_components=5, min_dist=0.0,
                  metric="cosine", random_state=42)

hdbscan_model = HDBSCAN(min_cluster_size=30, min_samples=10,
                        metric="euclidean",
                        cluster_selection_method="eom",
                        prediction_data=True)

topic_model = BERTopic(umap_model=umap_model, hdbscan_model=hdbscan_model)
topics, _ = topic_model.fit_transform(docs, embeddings=embeddings)
```

What changed → what it bought us

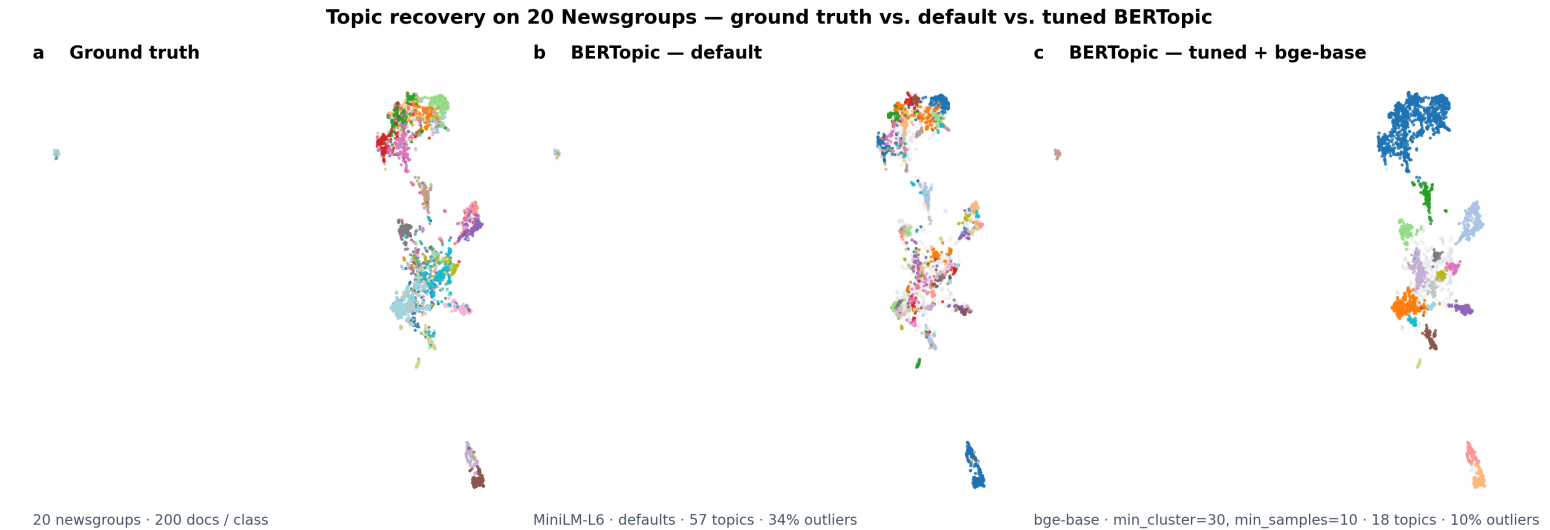
Vanilla vs. tuned, on the same 4 000-doc sample of 20 Newsgroups.

Encoder	all-MiniLM-L6-v2 (22 M params, 384-d)	BAAI/bge-base-en-v1.5 (110 M params, 768-d)
min_cluster_size	10 (default)	30 — fewer micro-clusters
min_samples	tied to min_cluster_size (default)	10 — set lower ⇒ fewer outliers
UMAP	defaults	defaults (already sensible)
Topics found	51	18 — matches the 20 true classes
Outliers (HDBSCAN -1)	34 %	10 %
Visual	fragmented over the true newsgroups	clusters trace the true classes

The encoder fixes **what gets grouped together**. The HDBSCAN knobs fix **how aggressively** it groups.

Numbers from the companion notebook ([src/bertopic.ipynb](#) , panels **b** and **c**). Subsample: 200 docs per newsgroup × 20 newsgroups, seed 42.

Vanilla vs. tuned — visualised

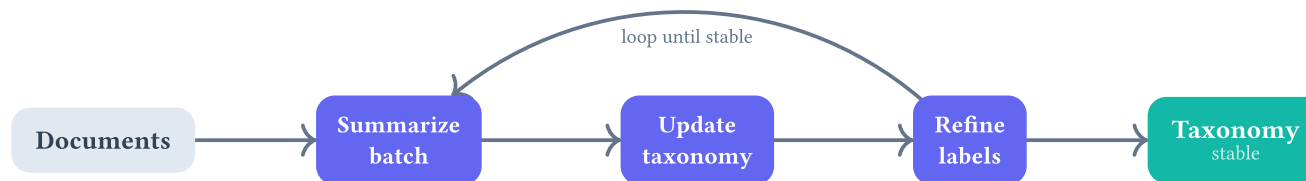


Each point is one document, plotted in a shared 2D UMAP fit on bge-base embeddings. Colour denotes (a) the human-assigned newsgroup, (b) the cluster found by vanilla BERTopic (MiniLM encoder, default UMAP/HDBSCAN), or (c) the cluster found after replacing the encoder with BAAI/bge-base-en-v1.5 and tuning HDBSCAN. Grey points are HDBSCAN outliers.

Generated by [src/bertopic.ipynb](#) — shared 2D UMAP fit on [bge-base-en-v1.5](#) embeddings. Grey points: HDBSCAN outliers.

Two modes of LLM topic extraction

INDUCTIVE
discover topics



DEDUCTIVE
assign to codebook



Inductive: LLM iteratively *summarises* → *updates* → *refines* a taxonomy from the corpus. **Deductive:** classify into a scheme you already have.

Inductive: Wan et al. 2024, *TnT-LLM: Text Mining at Scale with LLMs*, [arXiv:2403.12173](https://arxiv.org/abs/2403.12173) · Deductive: Pham et al. 2024, *TopicGPT*, NAACL, aclanthology.org/2024.naacl-long.164

The prompt *is* your codebook

Brief the model like a research assistant — start with the role, the task, and the codebook itself.

```
[Role]   You are coding documents about <domain>.

[Task]   Identify the topic(s) each document covers.
         Use a topic from the list below when one fits.
         If none of them does, propose a new one.

[Topics so far]
- price_concerns: cost, affordability, value for money
- reliability:    breakdowns, durability, trust in the product
- ...
```

Half of a good prompt is just being **explicit about what counts as a topic**. The list isn't decoration, it *is* your measurement instrument.

Anchor it with examples and rules

Definitions drift. Examples and bounded rules are what actually pin the model's behavior.

Examples

```
Doc: "way too expensive for a small battery"
  → price_concerns (existing)
Doc: "battery died in two months, warranty wouldn't cover it"
  → warranty: claims, coverage, support after purchase (new)
```

[Rules]

1. topics must be GENERAL, not document-specific
2. each topic captures a SINGLE idea, not a combination
3. new topics need a short label + one-line description
4. if nothing relevant is covered, return "None"

[Document]

```
{document}
```

A good prompt is **explicit, exemplified, and bounded** — and gives the model a way out when the codebook doesn't cover what's in front of it.

Structure adapted from Pham et al. 2024, *TopicGPT*, NAACL, aclanthology.org/2024.naacl-long.164 · `generate_topic` / `assign_topics` prompt templates

Make the output structured

Free text is unparseable and irreproducible. Pin a schema — every field should carry weight.

```
{
  "doc_id": "R0427",
  "topics": [
    {
      "label": "price_concerns",
      "source_quote": "way too expensive for a small battery",
      "clarity": 4,
      "rationale": "explicit cost-vs-value complaint, literal mention"
    },
    {
      "label": "reliability",
      "source_quote": "friend's one broke after a year",
      "clarity": 2,
      "rationale": "second-hand report, no time frame given"
    }
  ]
}
```

Not just a label dump. It should be a record you can **audit, validate, and aggregate** downstream.

What makes the schema load-bearing

Each field on the previous slide is there for a reason.

- **Pin it with the API** — JSON mode / function calling / structured-output; *validate* every record before downstream code touches it
- **Ground every label in a literal** `source_quote` copied from the document; auditability, and the model can't invent claims
- **Score on a small scale** (e.g. high, medium, low) instead of free-form confidence; a control signal for the pipeline, and a cognitive scaffold for the model.
- **Require a one-line** `rationale` a short free-text justification the model writes itself; improves the score and gives you debug context for free

Plus: in the system message, tell the model *"respond with valid JSON only, be terse, no commentary"* — verbosity is the #1 reason structured-output pipelines fail.

Schema pattern adapted from Riehl, Marin et al. 2026, *ARA: Agentic Reproducibility Assessment*, [arXiv:2605.02651](https://arxiv.org/abs/2605.02651) · schema-constrained JSON grounded in literal source quotes, ordinal scoring with rationales

Settings that matter

Four knobs that turn LLM-coding from a *thing that happened* into a *method*.

01 · PROMPT

Few-shot examples

A handful of labelled cases in the prompt anchors the codebook far better than definitions alone.

02 · DECODING

Temperature

Use 0 (or near-0) for coding tasks — you want consistency, not creativity.

03 · MODEL

Model choice

Bigger isn't always better. Test a cheap model first, escalate only if agreement is poor.

04 · METHOD

Pin everything

Model name, version/date, prompt, temperature — these **are** your method.

A ready framework: TopicGPT

TopicGPT (Pham et al., NAACL 2024) — a prompt-based pipeline that produces a **two-level topic hierarchy** and grounds every assignment in a **supporting quote**.



What you get out

- two-level hierarchy: high-level topics → subtopics
- every label paired with a **supporting quote** from the document
- topic list is editable — refine without retraining
- runs on OpenAI · Gemini · Azure · or any open model via vLLM

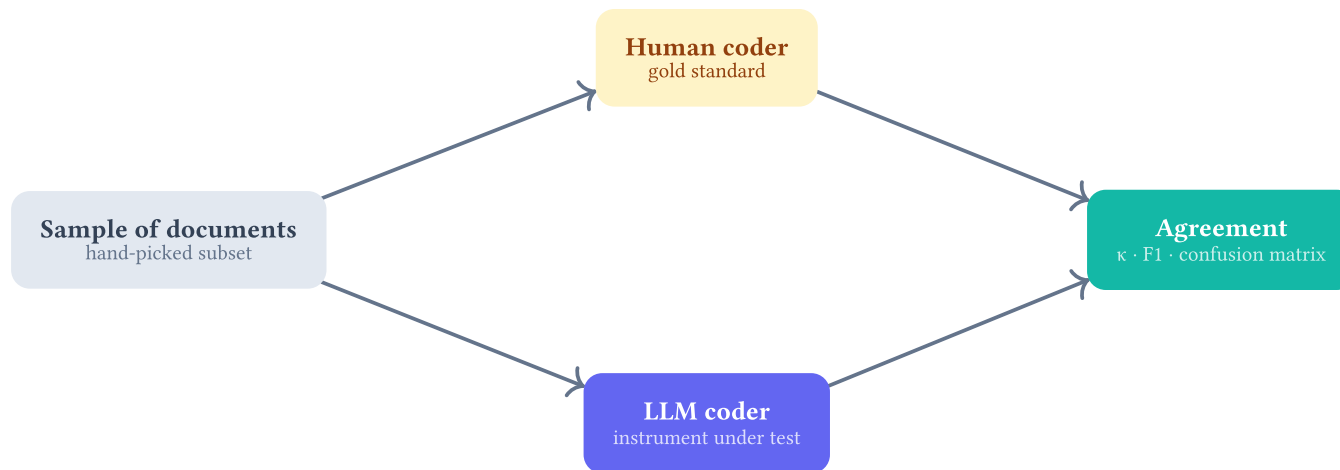
How it scored

- harmonic mean purity **0.74** vs. **0.64** for the strongest baseline, on human-annotated Wikipedia topics
- evaluated on **Wikipedia** and **Congressional Bills**
- also reports Adjusted Rand Index and NMI

Pham et al. 2024, *TopicGPT: A Prompt-based Topic Modeling Framework*, NAACL, aclanthology.org/2024.naacl-long.164 · code: github.com/chtomp223/topicGPT

Does it work? Treat the LLM as a coder

Validate it exactly as you would validate a human research assistant.



Hand-code a subset, compare, report agreement — same logic as inter-rater reliability.

The honest caveats

- **Reproducibility:** Approach 1 inherits BERTopic's; Approach 2 needs care: pin model, version, prompt, temperature
- **Prompt sensitivity:** wording shifts results; run a robustness check
- **Hallucinated topics:** constrain with a taxonomy; flag low confidence
- **Systematic bias:** does coding differ across respondent subgroups?
- **Measurement error:** topic labels used as variables → attenuation / bias downstream
- **Cost & access:** embedding vs. per-token cost; open vs. closed models

Stating the error budget is what makes it a *measurement* method.

When to use what

	Traditional BERTopic	Approach 1 — + LLM embeddings	Approach 2 — LLM only
Short / noisy / multilingual text	weak	strong	strong
Output	word lists	word lists	named + described
You still label by hand	yes	yes	no
Reproducibility	high	high-ish	needs care
Cost	low	embedding API	per-token API
Discovery vs. codebook	discovery	discovery	both

Often the answer: **Approach 1 to explore, Approach 2 to read.**

Resources to take home

Methods used in this deck

- *Topic models*: Blei, Ng & Jordan 2003, *LDA*, JMLR · Roberts, Stewart, Tingley et al. 2014, *STM*, AJPS · Grootendorst 2022, *BERTopic* · maartengr.github.io/BERTopic
- *Embeddings*: Reimers & Gurevych 2019, *Sentence-BERT*, EMNLP · Xiao et al. 2023, *bge / C-Pack* · [MTEB leaderboard](https://mteb.com/)

LLM-driven topic extraction

- Wan et al. 2024, *TnT-LLM: Text Mining at Scale with LLMs*, [arXiv:2403.12173](https://arxiv.org/abs/2403.12173)
- Pham et al. 2024, *TopicGPT*, NAACL, aclanthology.org/2024.naacl-long.164 · github.com/chtmp223/topicGPT

Structured output as measurement

- Riehl, Marin et al. 2026, *ARA: Agentic Reproducibility Assessment*, [arXiv:2605.02651](https://arxiv.org/abs/2605.02651)

Validation & rigour

- *LLMs as annotators*: Gilardi et al. 2023, *ChatGPT outperforms crowd workers...*, PNAS · Ziemis et al. 2024, *Can LLMs Transform Computational Social Science?*, Comp. Linguistics
- *Design-based inference from LLM labels*: Egami et al., *Design-Based Semi-Supervised Learning*
- *Open-source & reproducibility*: Spirling 2023, *Open-source generative AI for science*, Nature

Companion materials

- Walkthrough notebook: [src/bertopic.ipynb](https://src.bertopic.ipynb) — vanilla + tuned BERTopic on 20 Newsgroups, generates the comparison panel on slide 14
- Deck + code: github.com/AndresLaverdeMarin/llm-topic-extraction
- Dataset: [SetFit/20_newsgroups](https://huggingface.co/datasets/SetFit/20_newsgroups) on Hugging Face

Thank you

From **LLM as a component** to **LLM as the whole reader** — a calibratable instrument for turning text into topics.

Andres L. Marin · andres.laverde-marin@ec.europa.eu
Slides & notebook: github.com/AndresLaverdeMarin/llm-topic-extraction